

Ollama Model Fine-Tuning

Train Your Own PDF-Specialized Model

LoRA / QLoRA Fine-Tuning with Unsloth & MLX

Supports: NVIDIA GPUs (Unsloth) & Apple Silicon Macs (MLX)

Overview

This guide walks you through fine-tuning an LLM on your own PDF data and importing it into Ollama. Fine-tuning teaches the model new skills, domain-specific language, or response formats that prompt engineering alone cannot achieve.

The complete pipeline:

- Step 1: Collect PDFs and prepare training data (PDF -> JSONL)
- Step 2: Review and clean training data (most important step)
- Step 3: Fine-tune with LoRA (trains only ~1% of parameters)
- Step 4: Export to GGUF format
- Step 5: Import into Ollama and test

What is LoRA?

LoRA (Low-Rank Adaptation) freezes the original model and attaches small trainable adapter matrices to the attention layers. Instead of updating all 3 billion parameters, you only train ~30 million (about 1%). This means:

- 8GB GPU VRAM (or 16GB Mac RAM) is enough to fine-tune a 3B model
- Training takes minutes to hours, not days
- The adapter is only 20-50MB, not a full model copy

When to fine-tune vs. when to use RAG

Need	Use RAG	Use Fine-Tuning
Model should know my documents	YES	No
Model should answer in a specific format	No	YES
Model should understand domain jargon	Partially	YES
Best results	Combine	Combine

Step 0: Prerequisites

For NVIDIA GPU (Linux/Windows)

- NVIDIA GPU with 8GB+ VRAM (RTX 3060 or better)
- CUDA toolkit installed (nvidia-smi should show your GPU)
- Python 3.10+

```
# Verify GPU
nvidia-smi

# Install dependencies
cd ollama_fine_tuning
pip install -r requirements.txt
```

For Apple Silicon Mac (M1/M2/M4)

- Mac with Apple Silicon (M1, M2, M4 series)
- 16GB+ unified memory (24GB+ for 8B models)
- Python 3.10+

```
cd ollama_fine_tuning
pip install -r requirements_mac.txt
```

Estimated Training Times (500 examples, 3B model)

Hardware	Time	RAM/VRAM
RTX 3060 (12GB)	~15 min	8GB VRAM
RTX 3080 (10GB)	~10 min	8GB VRAM
RTX 4090 (24GB)	~5 min	8GB VRAM
Mac M1 (16GB)	~35 min	16GB RAM
Mac M2 Pro (32GB)	~20 min	16GB RAM
Mac M4 Pro (24GB)	~12 min	16GB RAM

Step 1: Prepare Training Data

The model cannot read PDFs directly. You must convert them into structured instruction/response pairs in JSONL format. Each line teaches the model: 'given this instruction and this input, produce this output.'

Data format

Each training example has three fields:

```
{"instruction": "Summarize this text.",  
  "input": "The patient was admitted on March 5...",  
  "output": "This section describes a patient admission..."}
```

The conversion pipeline

PDF -> Extract text -> Chunk into sections -> Pair with instructions -> JSONL

For NVIDIA GPU

```
# Put your PDFs in a folder  
mkdir pdfs  
cp ~/my-documents/*.pdf ./pdfs/  
  
# Generate training data (auto-fill responses with Ollama)  
python prepare_data.py --pdf-dir ./pdfs --auto-generate --model llama3.2  
  
# Or generate with placeholders to fill manually  
python prepare_data.py --pdf-dir ./pdfs
```

For Apple Silicon Mac

```
# Same PDF folder  
mkdir pdfs && cp ~/my-documents/*.pdf ./pdfs/  
  
# Generate MLX-format data (splits into train/valid/test)  
python prepare_data_mlx.py --pdf-dir ./pdfs --auto-generate --model llama3.2
```

TIP: The --auto-generate flag uses a running Ollama model to create responses. You still need to review them.

Step 2: Review and Clean Data

This is the MOST IMPORTANT step. The quality of your training data directly determines the quality of your fine-tuned model. Bad data = bad model.

What to do

- Open data/training.jsonl (or data/train.jsonl for MLX) in a text editor
- DELETE: Low-quality pairs (PDF headers, footers, table of contents, garbled text)
- FIX: Wrong or sloppy auto-generated responses
- ADD: Custom pairs for specific things you want the model to learn

Adding custom training examples

You can add your own domain-specific examples manually:

```
{"instruction": "What is our refund policy?",  
  "input": "",  
  "output": "Returns within 30 days with receipt..."}  
  
{"instruction": "Explain this medical term.",  
  "input": "Dyspnea",  
  "output": "Dyspnea means difficulty breathing..."}
```

How much data do you need?

Amount	Result
50-100 examples	Minimal effect, may overfit
100-500 examples	Noticeable improvement
500-2000 examples	Sweet spot for most use cases
2000+ examples	Strong specialization

Step 3: Fine-Tune the Model

NVIDIA GPU (Unsloth + QLoRA)

```
python train.py \  
  --data data/training.jsonl \  
  --base-model llama3.2-3b \  
  --epochs 3 \  
  --lora-rank 16
```

Apple Silicon Mac (MLX)

```
python train_mlx.py \  
  --base-model llama3.2-3b \  
  --iters 600 \  
  --batch-size 4 \  
  --lora-layers 16
```

Key parameters explained

Parameter	Default	What it does
base-model	llama3.2-3b	Model to fine-tune
epochs/iters	3 / 600	Passes over data
lora-rank	16	Adapter capacity (8-64)
batch-size	2-4	Examples per step
lr	2e-4 / 1e-5	Learning rate

Available base models

Name	Size	Min VRAM/RAM
llama3.2-1b	1B params	6GB / 8GB
llama3.2-3b	3B params	8GB / 16GB
llama3.1-8b	8B params	12GB / 24GB
mistral-7b	7B params	12GB / 24GB
phi-3.5	3.8B params	8GB / 16GB

WATCH THE LOSS: During training, the loss value should decrease. If it doesn't, your data may need more cleaning.

Step 4: Export to GGUF

GGUF is the format Ollama uses. This step merges your LoRA adapter back into the base model and quantizes it for efficient inference.

NVIDIA GPU

```
python export_gguf.py --quantization q4_k_m
```

Apple Silicon Mac

```
python export_mlx.py --base-model llama3.2-3b
```

Quantization options

Method	Size (7B)	Quality
q8_0	~8GB	Best quality
q6_k	~6GB	Very good
q5_k_m	~5GB	Good
q4_k_m	~4GB	Balanced (recommended)
q3_k_m	~3GB	Smallest usable

Step 5: Import into Ollama and Test

Create the Ollama model

```
# Import your GGUF into Ollama
ollama create my-pdf-model -f Modelfile

# Test it
ollama run my-pdf-model
```

The Modelfile

```
FROM ./model_gguf/unsloth.Q4_K_M.gguf    # or ./model.gguf for MLX

PARAMETER temperature 0.3
PARAMETER top_p 0.9

SYSTEM ""You are a specialized document analyst.
Answer precisely based on your training.""
```

Use in RAG system (optional)

Update .env in ollama_rag_pdf/ to use your fine-tuned model:

```
OLLAMA_MODEL=my-pdf-model
```

This gives you the best of both worlds: a model trained on your domain PLUS real-time retrieval from your documents.

Troubleshooting

CUDA out of memory

- Reduce batch-size to 1
- Use a smaller base model (llama3.2-1b)
- Reduce lora-rank to 8

Loss not decreasing

- Clean your training data (remove garbage examples)
- Increase epochs/iters
- Try a lower learning rate (1e-5)

Model outputs garbage after training

- You may have overfit - reduce epochs/iters
- Add more diverse training examples
- Lower the LoRA rank

MLX: Cannot allocate memory

- Close other applications to free RAM
- Use a smaller model (llama3.2-1b needs only 8GB)
- Reduce batch-size to 1

Quick Reference Card

NVIDIA GPU - Full Pipeline

```
# 1. Prepare data
python prepare_data.py --pdf-dir ./pdfs --auto-generate

# 2. Review: edit data/training.jsonl

# 3. Train
python train.py --data data/training.jsonl

# 4. Export
python export_gguf.py

# 5. Import
ollama create my-model -f Modelfile
ollama run my-model
```

Apple Silicon Mac - Full Pipeline

```
# 1. Prepare data
python prepare_data_mlx.py --pdf-dir ./pdfs --auto-generate

# 2. Review: edit data/train.jsonl

# 3. Train
python train_mlx.py

# 4. Export
python export_mlx.py

# 5. Import
ollama create my-model -f Modelfile
ollama run my-model
```

Project Structure

```
ollama_fine_tuning/  
  requirements.txt          # NVIDIA dependencies  
  requirements_mac.txt      # Apple Silicon dependencies  
  prepare_data.py          # PDF to JSONL (NVIDIA)  
  prepare_data_mlx.py      # PDF to JSONL (Mac)  
  train.py                 # Train with Unsloth (NVIDIA)  
  train_mlx.py             # Train with MLX (Mac)  
  export_gguf.py           # Export GGUF (NVIDIA)  
  export_mlx.py            # Export GGUF (Mac)  
  Modelfile                # Ollama import template  
  data/                   # Training data
```